UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

Ing. en Tecnologías de la Información e Innovación Digital

ACTIVIDAD PARCIAL 2

Búsqueda local

Hill Climbing y Simulated Annealing

Sección 3 · 2.3 Búsqueda local

Optimización de la ruta de un repartidor (TSP) con 21 ciudades

ALUMNA Erandy Naomy Rosas Toriz

GRUPO 171 · Cuatrimestre 7

ASIGNATURA Fundamentos de Inteligencia Artificial

DOCENTE Mtro. Javier García Lara

PERIODO 12 al 26 de octubre de 2026

FECHA DE ENTREGA 22 de octubre de 2026 · Individual

REPOSITORIO

I. Planteamiento del problema

Un repartidor debe visitar 21 ciudades del centro de México una sola vez y regresar al punto de partida. Es el **problema del viajante (TSP)**, un problema de optimización combinatoria: con 21 ciudades existen $(n-1)!/2 \approx 1.2 \times 10^{18}$ rutas distintas, así que revisarlas todas no es viable.

Los métodos de **búsqueda local** no recorren todo el espacio. Parten de una solución, generan soluciones vecinas con pequeños cambios y se mueven a la que mejora. Aquí se implementan y comparan dos de ellos: **Hill Climbing**, que solo acepta mejoras, y **Simulated Annealing**, que a veces acepta empeoramientos para escapar de óptimos locales (Russell & Norvig, 2020).

PREGUNTA GUÍA
¿Qué tan buena y qué tan consistente es la ruta que encuentra cada método, y cómo influyen sus parámetros (T, η e iteraciones)?

§ Objetivos

- § Definir el enunciado, la función objetivo, el estado y la vecindad del problema.
- § Implementar Hill Climbing (con y sin reinicios) y Simulated Annealing en Python.
- § Reportar la gráfica de convergencia, comparar resultados en 30 corridas y analizar la sensibilidad a los parámetros.

§ Metodología

Cada método se ejecutó 30 veces con semillas distintas (0 a 29) y la misma vecindad. Como referencia se incluye una ruta aleatoria. Para comprobar que los métodos funcionan, en una instancia reducida de 12 ciudades se calculó el óptimo exacto con programación dinámica (Held y Karp, 1962).

II. Formulación del problema

Elemento	Definición
Enunciado	Encontrar el orden de visita de 21 ciudades que minimice la distancia total del recorrido cerrado.
Estado	Permutación de las 21 ciudades; la ruta es cíclica (la última se une con la primera).
Función objetivo	f(ruta) = suma de las distancias entre ciudades consecutivas, incluido el regreso. Se **minimiza** (km).
Vecindad (2-opt)	Invertir un tramo de la ruta. Cada estado tiene $(n-1)(n-2)/2 =$ **190** vecinos.
Estado inicial	Una permutación aleatoria.
Criterio de paro	HC: ningún vecino mejora (óptimo local). SA: 20,000 iteraciones.

§ Parámetros de los algoritmos

Parámetro	Valor	Significado
T_0	50 km	Temperatura inicial: con T alta casi cualquier cambio se acepta.
η (factor de enfriamiento)	0.9995	En cada iteración $T \leftarrow T \cdot \eta$.
Iteraciones	20,000	Con estos valores la temperatura final es $T \approx 0.0023$.
Aceptación	$\exp(-\Delta / T)$	Probabilidad de aceptar un vecino peor que el actual (Δ = aumento en km).
Reinicios (HC)	20	HC se repite desde 20 rutas aleatorias y se conserva la mejor.

¿POR QUÉ 2-OPT?
Invertir un tramo cambia solo dos aristas de la ruta, así que el cambio de longitud se calcula en tiempo constante sin recalcular toda la ruta. Además elimina cruces entre aristas, que son los defectos más comunes de una mala ruta.

III. Datos y procedimiento

§ Datos de entrada

Ciudad	Lat.	Lon.	Ciudad	Lat.	Lon.
CDMX	19.43	-99.13	Tulancingo	20.08	-98.37
Texcoco	19.51	-98.88	Toluca	19.28	-99.66
Chalco	19.26	-98.90	Querétaro	20.59	-100.39
Amecameca	19.13	-98.77	San Juan del Río	20.39	-99.99
Atlautla	19.02	-98.77	Tula	20.05	-99.34
Cuautla	18.81	-98.95	Tepeji	19.90	-99.35
Cuernavaca	18.92	-99.23	Cholula	19.06	-98.30
Puebla	19.04	-98.21	Atlixco	18.91	-98.43
Tlaxcala	19.31	-98.24	Izúcar	18.60	-98.47
Apizaco	19.41	-98.14	Tehuacán	18.46	-97.39
Pachuca	20.10	-98.76			

Tabla 1. Coordenadas aproximadas (grados). La distancia entre dos ciudades se estima como la distancia geodésica (fórmula de Haversine) × 1.25, factor que representa la sinuosidad de la carretera. Son datos ilustrativos, no mediciones por carretera.

§ Procedimiento

§ Calcular la matriz de distancias de 21 × 21.

§ Generar una ruta inicial aleatoria.

§ Aplicar el algoritmo (HC, HC con reinicios o SA) con la vecindad 2-opt y registrar la longitud en cada paso.

§ Repetir 30 veces y comparar longitud final, desviación y tiempo.

§ Variar η para medir la sensibilidad de SA y verificar con el óptimo exacto en 12 ciudades.

```
Hill Climbing (máxima pendiente)        Simulated Annealing
ruta <- aleatoria                       ruta <- aleatoria ; T <- T0
repetir:                                repetir 20,000 veces:
    mejor <- mejor vecino 2-opt             v <- vecino 2-opt al azar
    si mejor no mejora: parar               d <- f(v) - f(ruta)
    ruta <- mejor                           si d < 0 o azar() < exp(-d/T):
devolver ruta                                   ruta <- v
                                            T <- T * eta
                                        devolver la mejor ruta vista
```

Pseudocódigo de los dos algoritmos (el código completo está en el Anexo A).

IV. Resultados

§ Comparación de los métodos (30 corridas)

Método	Mejor (km)	Media (km)	Desv. (km)	Peor (km)	Tiempo (s)
Ruta aleatoria	2,887.4	3,152.0	171.7	3,548.7	0.000
Hill Climbing	1,355.0	1,384.0	25.0	1,438.2	0.002
HC + 20 reinicios	1,355.0	1,355.3	0.6	1,357.1	0.045
Simulated Annealing	1,355.0	1,364.4	13.4	1,409.8	0.060

Tabla 2. Longitud de la ruta final en 30 corridas. Menor es mejor.

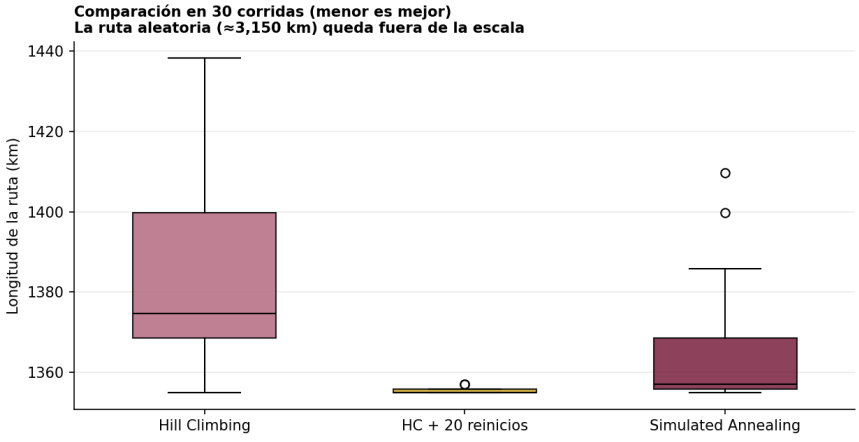


Figura 1. Distribución de las longitudes finales por método.

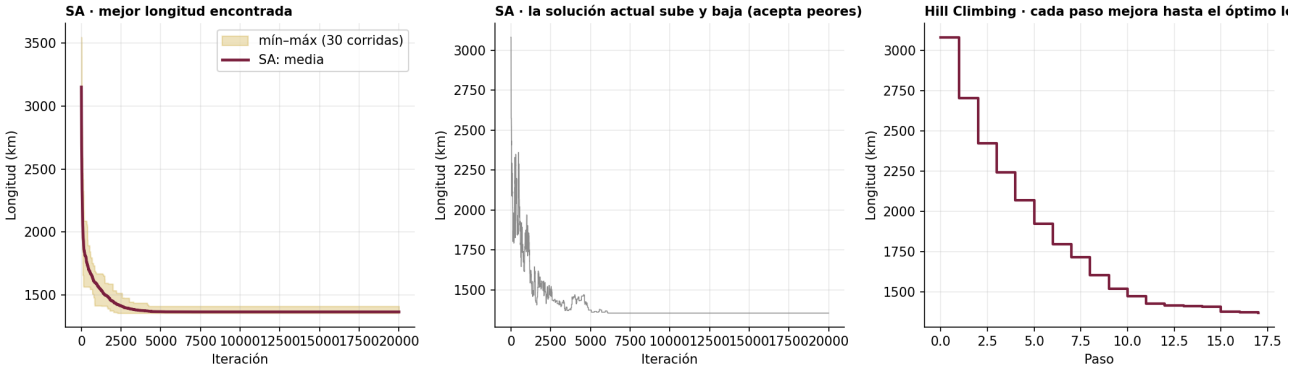


Figura 2. Convergencia: SA (media de 30 corridas y solución actual) y los pasos de una corrida de Hill Climbing.

§ Mejor ruta encontrada

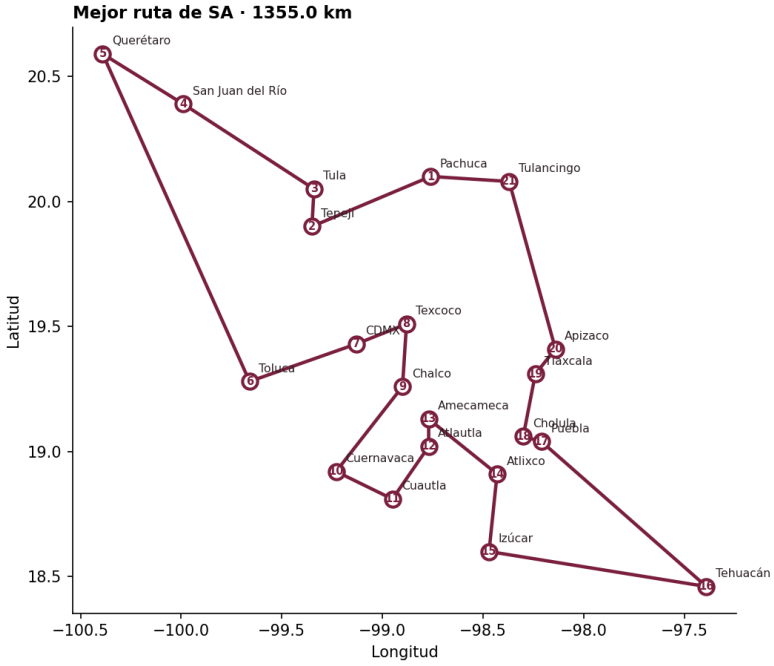


Figura 3. Mejor ruta de SA: 1355.0 km.

Orden de visita: Pachuca → Tepeji → Tula → San Juan del Río → Querétaro → Toluca → CDMX → Texcoco → Chalco → Cuernavaca → Cuautla → Atlautla → Amecameca → Atlixco → Izúcar → Tehuacán → Puebla → Cholula → Tlaxcala → Apizaco → Tulancingo → (regreso a Pachuca).

§ Sensibilidad de SA al factor de enfriamiento η

η	T final	Media (km)	Desv. (km)	Mejor (km)
0.99	0.0000	1,397.8	26.0	1,357.1
0.999	0.0000	1,375.0	19.0	1,355.0
0.9995	0.0023	1,367.0	17.3	1,355.0
0.9999	6.7661	1,358.2	4.5	1,355.0

Tabla 3. SA con $T_0 = 50$, 20,000 iteraciones y 15 corridas por valor de η.

§ Verificación con el óptimo exacto (12 ciudades)

Para las primeras 12 ciudades de la Tabla 1, el óptimo exacto (Held-Karp) es **593.6 km**. En 15 corridas, **SA llegó al óptimo 15 veces** y Hill Climbing sin reinicios solo 7 veces.

V. Análisis y comparación

§ **Hill Climbing se queda en óptimos locales.** Termina cuando ningún vecino mejora; por eso su media (1,384.0 km) es peor que la mejor ruta hallada (1,355.0 km) y varía entre corridas (desv. 25.0 km).

§ **SA puede escapar de ellos.** Con T alta acepta empeoramientos: un vecino 20 km peor se acepta con probabilidad 0.67 si T = 50, pero solo 2.1e-09 si T = 1. Así explora al inicio y se vuelve voraz al final.

§ **Los reinicios compensan a Hill Climbing en esta instancia.** Con 20 reinicios su media fue 1,355.3 km frente a 1,364.4 km de SA con η = 0.9995. Con solo 21 ciudades el paisaje de soluciones es sencillo y 20 arranques aleatorios bastan; no se puede afirmar que SA sea mejor aquí.

§ **El enfriamiento lento ayuda a SA.** Subir η de 0.99 a 0.9999 redujo la media de 1,397.8 a 1,358.2 km y la desviación de 26.0 a 4.5 km.

§ **Costo por paso.** Un paso de Hill Climbing evalúa los 190 vecinos (cada uno en O(1)) y aplica el mejor en O(n). Una iteración de SA evalúa un vecino en O(1) y, si lo acepta, lo aplica en O(n). Memoria: O(n) en ambos.

§ **Alcance de la verificación.** El óptimo exacto solo se comprobó con 12 ciudades. Para 21 ciudades 1,355.0 km es la mejor ruta encontrada por los tres métodos, pero no se demostró que sea óptima.

VI. Conclusiones reflexivas

§ La búsqueda local permite resolver en milisegundos un problema con más de 10^{18} soluciones, a cambio de no garantizar el óptimo.

§ Elegir bien la vecindad importa tanto como el algoritmo: 2-opt es barata de evaluar y corrige cruces de la ruta.

§ Ningún método ganó en todo. Hill Climbing con reinicios fue rápido y consistente en una instancia pequeña; SA fue más sensible a sus parámetros, pero con enfriamiento lento se acercó al mejor resultado.

§ Las comparaciones deben repetirse muchas veces: una sola corrida de SA puede dar un resultado bueno o malo por azar.

§ Limitaciones: los datos son ilustrativos y la instancia es pequeña. En problemas con miles de puntos o restricciones (horarios, capacidades) la diferencia entre métodos sería más marcada, y eso se explora con los algoritmos genéticos de la S6.

VII. Declaración de uso de inteligencia artificial

Herramienta	Claude (Anthropic, 2026), modelo Claude Sonnet 5.5, con ejecución de código Python.
Prompt (resumen)	Se entregó la hoja de la Actividad Parcial 2 y se pidió el código y el PDF de la sección 3 (2.3 Búsqueda local con Hill Climbing y Simulated Annealing).
Uso	Diseño del problema y de los experimentos, escritura del código (Hill Climbing, Simulated Annealing, Held-Karp, gráficas), borrador del texto y maquetación del PDF (reportlab).
Verificación	El código se ejecutó completo y los resultados se contrastaron con el óptimo exacto en 12 ciudades. Los datos geográficos son aproximados. Las referencias deben cotejarse con las fuentes originales antes de entregar.

R. Referencias (APA 7.ª ed.)

Anthropic. (2026). *Claude Sonnet 5.5* [Modelo de lenguaje de gran tamaño]. https://claude.ai

Croes, G. A. (1958). A method for solving traveling-salesman problems. *Operations Research, 6*(6), 791-812.

Held, M., & Karp, R. M. (1962). A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics, 10*(1), 196-210.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90-95. https://doi.org/10.1109/MCSE.2007.55

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science, 220*(4598), 671-680. https://doi.org/10.1126/science.220.4598.671

Russell, S., & Norvig, P. (2020). *Inteligencia Artificial: Un enfoque moderno* (4.ª ed.). Pearson.

Wirsansky, E. (2020). *Hands-on genetic algorithms with Python*. Packt.

A. Anexo A. Código Python

Archivo: `s3_hill_climbing_sa.py`

```python
"""S3 · Búsqueda local: Hill Climbing y Simulated Annealing (2.3)

Problema: ruta mínima de un repartidor que visita 21 ciudades del centro de México una sola vez
y regresa al punto de partida (TSP). Se implementan:
  · Hill Climbing de máxima pendiente (con y sin reinicios aleatorios)
  · Simulated Annealing (recocido simulado)
Ambos usan la misma vecindad: el movimiento 2-opt (invertir un tramo de la ruta).

  Función objetivo: longitud total del recorrido en km (se MINIMIZA)
  Estado          : permutación de las 21 ciudades
  Vecindad        : todas las rutas que resultan de invertir un tramo (2-opt)
  Parámetros SA   : T0 (temperatura inicial), eta (factor de enfriamiento), iteraciones

Ejecutar: python s3_hill_climbing_sa.py
Requiere: pip install numpy matplotlib
Genera:   figuras/convergencia.png, figuras/comparacion.png, figuras/mejor_ruta.png y resultados_s3.json
"""
import itertools
import json
import math
import os
import random
import time

import matplotlib
matplotlib.use("Agg")  # genera PNG sin abrir ventana
import matplotlib.pyplot as plt
import numpy as np

AQUI = os.path.dirname(os.path.abspath(__file__))
FIG = os.path.join(AQUI, "figuras")
os.makedirs(FIG, exist_ok=True)

# ======================================================================
# 1. Datos (coordenadas aproximadas) y matriz de distancias
# ======================================================================
CIUDADES = {
    "CDMX": (19.43, -99.13), "Texcoco": (19.51, -98.88), "Chalco": (19.26, -98.90),
    "Amecameca": (19.13, -98.77), "Atlautla": (19.02, -98.77), "Cuautla": (18.81, -98.95),
    "Cuernavaca": (18.92, -99.23), "Puebla": (19.04, -98.21), "Tlaxcala": (19.31, -98.24),
    "Apizaco": (19.41, -98.14), "Pachuca": (20.10, -98.76), "Tulancingo": (20.08, -98.37),
    "Toluca": (19.28, -99.66), "Querétaro": (20.59, -100.39), "San Juan del Río": (20.39, -99.99),
    "Tula": (20.05, -99.34), "Tepeji": (19.90, -99.35), "Cholula": (19.06, -98.30),
    "Atlixco": (18.91, -98.43), "Izúcar": (18.60, -98.47), "Tehuacán": (18.46, -97.39),
}
NOMBRES = list(CIUDADES)
SINUOSIDAD = 1.25   # distancia por carretera estimada = línea recta x 1.25


def haversine(a, b):
    la1, lo1, la2, lo2 = map(math.radians, (*a, *b))
    h = math.sin((la2 - la1) / 2) ** 2 + math.cos(la1) * math.cos(la2) * math.sin((lo2 - lo1) / 2) ** 2
    return 2 * 6371.0 * math.asin(math.sqrt(h))


def matriz_distancias(nombres):
    n = len(nombres)
    D = np.zeros((n, n))
    for i, j in itertools.combinations(range(n), 2):
        D[i, j] = D[j, i] = haversine(CIUDADES[nombres[i]], CIUDADES[nombres[j]]) * SINUOSIDAD
    return D


# ======================================================================
# 2. Función objetivo, vecindad 2-opt y utilidades
# ======================================================================
def longitud(ruta, D):
    return sum(D[ruta[i], ruta[(i + 1) % len(ruta)]] for i in range(len(ruta)))


```

```
71  def delta_2opt(ruta, i, j, D):
72      """Cambio de longitud al invertir el tramo ruta[i..j] (solo se recalculan 2 aristas)."""
73      n = len(ruta)
74      a, b = ruta[i - 1], ruta[i]
75      c, d = ruta[j], ruta[(j + 1) % n]
76      return D[a, c] + D[b, d] - D[a, b] - D[c, d]
77
78
79  def aplicar_2opt(ruta, i, j):
80      return ruta[:i] + ruta[i:j + 1][::-1] + ruta[j + 1:]
81
82
83  def ruta_aleatoria(n, rng):
84      r = list(range(n))
85      rng.shuffle(r)
86      return r
87
88
89  # ======================================================================
90  # 3. Hill Climbing de máxima pendiente
91  # ======================================================================
92  def hill_climbing(D, rng, ruta=None):
93      """Evalúa TODA la vecindad 2-opt, se mueve al mejor vecino y se detiene en un óptimo local."""
94      n = len(D)
95      ruta = ruta or ruta_aleatoria(n, rng)
96      hist = [longitud(ruta, D)]
97      evals = 0
98      while True:
99          mejor_d, mejor_mov = 0.0, None
100         for i in range(1, n - 1):
101             for j in range(i + 1, n):
102                 d = delta_2opt(ruta, i, j, D)
103                 evals += 1
104                 if d < mejor_d - 1e-9:
105                     mejor_d, mejor_mov = d, (i, j)
106         if mejor_mov is None:
107             break                                # óptimo local: ningún vecino mejora
108         ruta = aplicar_2opt(ruta, *mejor_mov)
109         hist.append(hist[-1] + mejor_d)
110     return ruta, hist[-1], hist, evals
111
112
113 def hill_climbing_reinicios(D, rng, reinicios=20):
114     mejor, mejor_l, curva, evals = None, math.inf, [], 0
115     for _ in range(reinicios):
116         r, l, _, e = hill_climbing(D, rng)
117         evals += e
118         if l < mejor_l:
119             mejor, mejor_l = r, l
120         curva.append(mejor_l)
121     return mejor, mejor_l, curva, evals
122
123
124 # ======================================================================
125 # 4. Simulated Annealing
126 # ======================================================================
127 def simulated_annealing(D, rng, T0=50.0, eta=0.9995, iteraciones=20000):
128     """Acepta siempre los vecinos mejores y los peores con probabilidad exp(-delta/T).
129     La temperatura baja en cada iteración: T <- T * eta."""
130     n = len(D)
131     ruta = ruta_aleatoria(n, rng)
132     actual = longitud(ruta, D)
133     mejor, mejor_l = ruta[:], actual
134     T = T0
135     curva_mejor, curva_actual = [mejor_l], [actual]
136     for _ in range(iteraciones):
137         i, j = sorted(rng.sample(range(1, n), 2))
138         d = delta_2opt(ruta, i, j, D)
139         if d < 0 or rng.random() < math.exp(-d / T):
140             ruta = aplicar_2opt(ruta, i, j)
141             actual += d
142             if actual < mejor_l - 1e-9:
143                 mejor, mejor_l = ruta[:], actual
144         T *= eta
145         curva_mejor.append(mejor_l)
146         curva_actual.append(actual)
147     return mejor, mejor_l, curva_mejor, curva_actual
148
```

```
149
150 # ======================================================================
151 # 5. Verificación: óptimo exacto en una instancia pequeña (Held-Karp)
152 # ======================================================================
153 def optimo_exacto(D):
154     """Programación dinámica de Held-Karp: óptimo del TSP para n pequeño (aquí n = 12)."""
155     n = len(D)
156     INF = float("inf")
157     dp = {(1 << k, k): D[0, k] for k in range(1, n)}
158     for tam in range(2, n):
159         for sub in itertools.combinations(range(1, n), tam):
160             mask = sum(1 << k for k in sub)
161             for k in sub:
162                 prev = mask ^ (1 << k)
163                 dp[(mask, k)] = min(dp[(prev, m)] + D[m, k] for m in sub if m != k)
164     full = sum(1 << k for k in range(1, n))
165     return min(dp[(full, k)] + D[k, 0] for k in range(1, n))
166
167
168 # ======================================================================
169 # 6. Experimentos
170 # ======================================================================
171 def experimento(D, corridas=30):
172     res = {"aleatorio": [], "hc": [], "hc_reinicios": [], "sa": []}
173     tiempos = {k: [] for k in res}
174     curvas_sa = []
175     for s in range(corridas):
176         rng = random.Random(s)
177         t = time.perf_counter(); r = ruta_aleatoria(len(D), rng); res["aleatorio"].append(longitud(r, D)); tiempos["aleatorio"].appen
178         t = time.perf_counter(); res["hc"].append(hill_climbing(D, random.Random(s))[1]); tiempos["hc"].append(time.perf_counter() -
179         t = time.perf_counter(); res["hc_reinicios"].append(hill_climbing_reinicios(D, random.Random(s))[1]); tiempos["hc_reinicios"]
180         t = time.perf_counter(); _, l, cm, _ = simulated_annealing(D, random.Random(s)); res["sa"].append(l); tiempos["sa"].append(ti
181         curvas_sa.append(cm)
182     return res, tiempos, np.array(curvas_sa)
183
184
185 def sensibilidad_eta(D, etas=(0.99, 0.999, 0.9995, 0.9999), corridas=15):
186     filas = []
187     for e in etas:
188         f = [simulated_annealing(D, random.Random(s), eta=e)[1] for s in range(corridas)]
189         filas.append({"eta": e, "T_final": 50.0 * e ** 20000, "media": float(np.mean(f)), "desv": float(np.std(f)), "mejor": float(np
190     return filas
191
192
193 # ======================================================================
194 # 7. Gráficas
195 # ======================================================================
196 VINO, ORO, GRIS, TINTA = "#7A1F3D", "#C9A227", "#8C8C8C", "#2B2024"
197
198
199 def graficas(D, res, curvas_sa, mejor_ruta, hc_hist, hc_reinicios_curva, sa_actual):
200     # (a) convergencia
201     fig, ax = plt.subplots(1, 3, figsize=(14, 4.2))
202     x = np.arange(curvas_sa.shape[1])
203     ax[0].fill_between(x, curvas_sa.min(0), curvas_sa.max(0), color=ORO, alpha=0.3, label="mín–máx (30 corridas)")
204     ax[0].plot(x, curvas_sa.mean(0), color=VINO, lw=2.2, label="SA: media")
205     ax[0].set_title("SA · mejor longitud encontrada", fontsize=10, fontweight="bold", loc="left")
206     ax[0].set_xlabel("Iteración"); ax[0].set_ylabel("Longitud (km)"); ax[0].legend()
207     ax[1].plot(sa_actual, color=GRIS, lw=0.7, label="solución actual")
208     ax[1].set_title("SA · la solución actual sube y baja (acepta peores)", fontsize=10, fontweight="bold", loc="left")
209     ax[1].set_xlabel("Iteración"); ax[1].set_ylabel("Longitud (km)")
210     ax[2].step(range(len(hc_hist)), hc_hist, color=VINO, lw=2, where="post", label="HC (una corrida)")
211     ax[2].set_title("Hill Climbing · cada paso mejora hasta el óptimo local", fontsize=10, fontweight="bold", loc="left")
212     ax[2].set_xlabel("Paso"); ax[2].set_ylabel("Longitud (km)")
213     for a in ax:
214         a.spines[["top", "right"]].set_visible(False); a.grid(alpha=0.25)
215     fig.tight_layout(); fig.savefig(os.path.join(FIG, "convergencia.png"), dpi=150); plt.close(fig)
216     # (b) comparación
217     fig, ax = plt.subplots(figsize=(8.4, 4.4))
218     etiquetas = ["Hill Climbing", "HC + 20 reinicios", "Simulated Annealing"]
219     datos = [res["hc"], res["hc_reinicios"], res["sa"]]
220     bp = ax.boxplot(datos, patch_artist=True, tick_labels=etiquetas, widths=0.55)
221     for patch, c in zip(bp["boxes"], ["#B56A82", ORO, VINO]):
222         patch.set_facecolor(c); patch.set_alpha(0.85)
223     for m in bp["medians"]:
224         m.set_color("black")
225     ax.set_ylabel("Longitud de la ruta (km)")
226     ax.set_title("Comparación en 30 corridas (menor es mejor)\nLa ruta aleatoria (≈3,150 km) queda fuera de la escala", fontsize=10,
```

```
    ax.spines[["top", "right"]].set_visible(False); ax.grid(axis="y", alpha=0.25)
    fig.tight_layout(); fig.savefig(os.path.join(FIG, "comparacion.png"), dpi=150); plt.close(fig)
    # (c) mejor ruta
    fig, ax = plt.subplots(figsize=(7.2, 6.2))
    pts = [CIUDADES[NOMBRES[i]] for i in mejor_ruta] + [CIUDADES[NOMBRES[mejor_ruta[0]]]]
    ax.plot([p[1] for p in pts], [p[0] for p in pts], "-", color=VINO, lw=2.2, zorder=1)
    for k, i in enumerate(mejor_ruta):
        la, lo = CIUDADES[NOMBRES[i]]
        ax.scatter(lo, la, s=95, color="white", edgecolor=VINO, linewidth=2, zorder=3)
        ax.text(lo, la, str(k + 1), ha="center", va="center", fontsize=7, fontweight="bold", color=VINO, zorder=4)
        ax.annotate(NOMBRES[i], (lo, la), textcoords="offset points", xytext=(6, 6), fontsize=7.5, color=TINTA)
    ax.set_title(f"Mejor ruta de SA · {longitud(mejor_ruta, D):.1f} km", fontsize=11, fontweight="bold", loc="left")
    ax.set_xlabel("Longitud"); ax.set_ylabel("Latitud"); ax.spines[["top", "right"]].set_visible(False)
    fig.tight_layout(); fig.savefig(os.path.join(FIG, "mejor_ruta.png"), dpi=150); plt.close(fig)


def main():
    D = matriz_distancias(NOMBRES)
    n = len(NOMBRES)
    res, tiempos, curvas_sa = experimento(D)
    print(f"=== TSP con {n} ciudades · 30 corridas por método ===")
    print(f"{'Método':22}{'Mejor':>9}{'Media':>9}{'Desv.':>8}{'Peor':>9}{'Tiempo (s)':>12}")
    nombres = {"aleatorio": "Ruta aleatoria", "hc": "Hill Climbing", "hc_reinicios": "HC + 20 reinicios", "sa": "Simulated Annealing"
    for k, v in res.items():
        print(f"{nombres[k]:22}{min(v):>9.1f}{np.mean(v):>9.1f}{np.std(v):>8.1f}{max(v):>9.1f}{np.mean(tiempos[k]):>12.3f}")

    # corrida de ejemplo para las gráficas
    _, _, hc_hist, _ = hill_climbing(D, random.Random(0))
    _, _, hc_curva, _ = hill_climbing_reinicios(D, random.Random(0))
    mejor_sa, l_sa, _, sa_actual = simulated_annealing(D, random.Random(int(np.argmin(res["sa"]))))
    print("Mejor ruta SA:", " -> ".join(NOMBRES[i] for i in mejor_sa), f"-> {l_sa:.1f} km")

    sens = sensibilidad_eta(D)
    print("\nSensibilidad al factor de enfriamiento eta (T0=50, 20,000 iteraciones, 15 corridas)")
    for f in sens:
        print(f"eta={f['eta']:<7} T_final={f['T_final']:8.4f}  media={f['media']:.1f}  desv={f['desv']:.1f}  mejor={f['mejor']:.1f}")

    # verificación con el óptimo exacto en las primeras 12 ciudades
    sub = NOMBRES[:12]
    Ds = matriz_distancias(sub)
    opt = optimo_exacto(Ds)
    sa12 = [simulated_annealing(Ds, random.Random(s), iteraciones=8000)[1] for s in range(15)]
    hc12 = [hill_climbing(Ds, random.Random(s))[1] for s in range(15)]
    print(f"\nVerificación con 12 ciudades: óptimo exacto (Held-Karp) = {opt:.1f} km")
    print(f"  SA llegó al óptimo en {sum(abs(x - opt) < 1e-6 for x in sa12)}/15 corridas; HC en {sum(abs(x - opt) < 1e-6 for x in hc1

    graficas(D, res, curvas_sa, mejor_sa, hc_hist, hc_curva, sa_actual)
    with open(os.path.join(AQUI, "resultados_s3.json"), "w", encoding="utf-8") as f:
        json.dump({"res": res, "tiempos": {k: float(np.mean(v)) for k, v in tiempos.items()}, "sens": sens,
                   "mejor_sa": [NOMBRES[i] for i in mejor_sa], "l_sa": l_sa, "hc_pasos": len(hc_hist) - 1,
                   "verif": {"n": 12, "opt": opt, "sa_ok": int(sum(abs(x - opt) < 1e-6 for x in sa12)),
                             "hc_ok": int(sum(abs(x - opt) < 1e-6 for x in hc12))}}, f, ensure_ascii=False, indent=1)


if __name__ == "__main__":
    main()
```

B. Anexo B. Salida de la ejecución

```
=== TSP con 21 ciudades · 30 corridas por método ===
Método                    Mejor    Media   Desv.     Peor  Tiempo (s)
Ruta aleatoria           2887.4   3152.0   171.7   3548.7       0.000
Hill Climbing            1355.0   1384.0    25.0   1438.2       0.002
HC + 20 reinicios        1355.0   1355.3     0.6   1357.1       0.045
Simulated Annealing      1355.0   1364.4    13.4   1409.8       0.060
Mejor ruta SA: Pachuca -> Tepeji -> Tula -> San Juan del Río -> Querétaro -> Toluca -> CDMX -> Texcoco -> Chalco -> Cuernavaca -> Cuautla

Sensibilidad al factor de enfriamiento eta (T0=50, 20,000 iteraciones, 15 corridas)
eta=0.99    T_final=  0.0000  media=1397.8  desv=26.0  mejor=1357.1
eta=0.999   T_final=  0.0000  media=1375.0  desv=19.0  mejor=1355.0
eta=0.9995  T_final=  0.0023  media=1367.0  desv=17.3  mejor=1355.0
eta=0.9999  T_final=  6.7661  media=1358.2  desv=4.5  mejor=1355.0

Verificación con 12 ciudades: óptimo exacto (Held-Karp) = 593.6 km
  SA llegó al óptimo en 15/15 corridas; HC en 7/15
```